

Ahmed Corrugation Machines

Two attempts at a factory management system exist on this machine. Neither ever ran in the factory. The business logic inside them is sound and worth salvaging — the way it was built is what stopped it from shipping.

12,007 lines Python PyQt5 + Excel Last touched Sep 2022 Never deployed

AT A GLANCE

What the numbers say

29

Excel sheets acting as the database, one per customer

364

Hardcoded paper-type names across four files

376

global statements holding state together

127

Direct read/write calls against the spreadsheet

0

Automated tests, backups, or user accounts

The system was built as a Windows desktop application in 2022 using PyQt5, with two Excel workbooks — `book.xlsx` and `BookExpense.xlsx` — standing in for a database. Every screen reads the entire workbook into memory, mutates it, and writes the whole file back.

One correction to the folder names before anything else: **“Factory Management System First Deployment” is the newer and more complete version**, despite what “Final” suggests. It was last edited 24 September 2022, six days after “Final”, and it alone contains the Google Drive backup scripts and the stock-record generator. Any rebuild should

treat it as the reference, not the other folder.

INVENTORY

What the system actually does

MODULE	FILE	LINES	RESPONSIBILITY
Customers & ledger	factorycustomers.py	3,831	Client register, per-customer ledger, add and delete bills, search by ID or name, customer PDF statement
Cash bill	CashBillclass.py	2,966	Walk-in billing across seven item forms, live running total, receipt PDF, WhatsApp send, stock deduction
Stock	stock.py	1,917	Add, search and delete stock for rolls, reels and totay; per-paper-type quantity tables
Daily report	dailyreportv2.py	865	Day's debit, credit and expenses; cash vs client breakdown; net position; PDF report
Stock in/out	stock_in_out_record.py	454	Movement history per stock form, filterable, with PDF export
Expenses	expense.py	400	Daily expense lines concatenated into one row with a running total

WORTH KEEPING

The domain model is the real asset

Strip away the Qt code and what remains is a well-observed model of how the business actually trades. This is the part that took real knowledge to work out, and it transfers directly to a web rebuild.

SALE FORM	PRICED ON	STOCK EFFECT
Rolls	Quantity × rate, by paper type and size	Decrements roll count for that type and size
Reels	Weight (kg) × rate	Removes the matched reel from stock entirely
Packets	Sheet area and grammage — see formula below	Consumes a reel or a tota, depending on source
Totay	Quantity × weight × rate	Decrements tota stock
Jutta	Weight (kg) × rate	Not stock-tracked
Raddi	Weight (kg) × rate	Not stock-tracked
Nali	Weight (kg) × rate	Not stock-tracked

PACKET PRICING

`price = (length × width × grammage ÷ 15500) × packets × rate`

The standard ream-weight conversion. This single line is the most valuable thing in the codebase — it encodes how the factory converts sheet dimensions into a sellable weight, and it must carry over exactly.

Paper catalogue

Nine base types, each with a *Bareek* (thin) variant, giving eighteen in total:

- Fluting · Super Fluting
- L1 · L2
- Test Liner
- Box Board 2.5 No · Box Board 3 No
- Local Kraft · Imported Kraft

Sold in sizes 17–22, each carrying its own rate and description.

Customer model

- **Cash customers** — IDs prefixed `c` , identified by name plus contact number so duplicates stay separate
- **Ledger clients** — plain integer IDs, each with a running account
- Every entry carries date, receipt number, details, **debit**, **credit**, credit method, **rent**, and a running **balance**
- Ledger clients also record the page number in the paper register, so the two can be reconciled

DIAGNOSIS

Why it never reached the factory

A spreadsheet is being used as a database

CRITICAL

Customer ledgers live one-sheet-per-customer. Adding a client adds a sheet; adding a bill rewrites the entire workbook. There is no transaction, no locking, and no recovery — a crash mid-save loses everything. Your own `readme.txt` records the outcome: *“xls file is corrupted and causing issues while saving updating forms data.”* That was never a bug to fix; it is the guaranteed result of the design.

```
29 sheets in book.xlsx  127 read/write calls  no backup path
```

Adding one new product means editing hundreds of places

CRITICAL

Paper types are not data — they are written into the source code as literal text, 364 times across four files. `stock.py` alone contains eighteen near-identical functions (`flutingtable()`, `l1table()`, `boxboard3table()` ...) that differ only by which name they mention. Billing repeats the same eighteen-branch chain again for stock deduction.

This matters more than anything else here, because you’ve told me **new products are coming**. Under this design, each one is a multi-day edit across the whole codebase with a high chance of missing a branch. In the rebuild, a new product is a row in a table and takes thirty seconds.

```
stock.py: 132  factorycustomers.py: 104  CashBillclass.py: 102
stock_in_out_record.py: 26
```

It cannot be installed, let alone reached from a phone

CRITICAL

It is a Windows-only desktop program with a window pinned to `setFixedSize(1360, 850)` and every control placed at fixed pixel coordinates. PDF generation depends on a bundled `poppler-0.68.0` Windows binary from 2018. The dependency list pins PyQt5 against a Python version now four years old, and includes a package literally named `module`. Getting it running on a new machine in 2026 is itself a project.

```
fixed 1360x850 layout  poppler win32 binaries  PyQt5, pywhatkit, pdf2image
```

Receipt numbers can collide

CORRECTNESS

The next receipt number is produced by reading the last row of the sheet and adding one, with cash bills and client bills drawing from the same counter. Two bills raised close together — or one raised while the workbook is open elsewhere — produce duplicate receipt numbers on real customer paperwork. A database sequence removes this class of bug permanently.

Business rules are buried inside screen code

MAINTAINABILITY

Pricing, validation, stock deduction and file writing all live inside button handlers nested within UI setup functions, coordinated by 376 `global` statements. `factorycustomers.py` is a single 3,831-line file. Nothing can be tested, reused, or changed in isolation — which is why small changes kept breaking distant features.

WhatsApp sending is inherently fragile

RELIABILITY

`pywhatkit` works by opening WhatsApp Web in a browser and simulating keystrokes at a scheduled time. It breaks whenever WhatsApp changes its interface, requires the screen to be unlocked and untouched, and cannot report whether the message was delivered. A `wa.me` link — or the official WhatsApp Business API if automated sending matters — replaces it with something dependable.

Two divergent copies with misleading names

HOUSEKEEPING

The two folders have drifted apart — “First Deployment” is ahead by roughly 400 lines and holds features the other lacks, while “Final” is the older snapshot. With no version control, there is no record of what changed or why. The rebuild starts in Git on day one.

SALVAGE

What carries over, what gets rebuilt

KEEP

REBUILD

- The seven sale forms and how each is priced
 - The packet ream-weight formula
 - Cash-customer vs ledger-client distinction and ID scheme
 - Debit / credit / rent / running balance ledger structure
 - The eighteen paper types, sizes, rates and descriptions
 - Stock movement semantics — what a sale does to which stock
 - Report content: what belongs on a daily report and a receipt
 - Manual-ledger page reference, for reconciling against paper
- All 12,000 lines of PyQt5 interface code
 - Excel as storage — replaced by a real database
 - Hardcoded product lists — replaced by editable catalogue tables
 - reportlab + poppler PDFs — replaced by server-side generation
 - pywhatkit automation
 - The Google Drive upload scripts — superseded by managed backups
 - Receipt numbering logic

TARGET

The shape of the web system

Twenty-nine spreadsheet sheets collapse into roughly seven tables. Adding a customer no longer changes the structure of anything; adding a product is a row, not a release.

TABLE	REPLACES	NOTES
customers	Sheets 1 ... 4 plus the cash roster	One row per customer; cash and ledger distinguished by a type column, not by storage location
ledger_entries	cash table + every per-customer sheet	Balance is calculated on read, never stored — so it can never drift
item_types	18 hardcoded names in 4 files	Editable in the app. This is what makes new products trivial
stock_items	reels_stock , rolls_stock , totay , and 18 per-type sheets	Type, size, unit, quantity or weight, rate

stock_movements	The three *_stock_in_out sheets	Every in and out, linked to the bill that caused it
expenses	BookExpense.xlsx	One row per expense line rather than text concatenated into a cell
users	Nothing — did not exist	Logins and roles, so you can see who entered what

Daily reports stop being a stored sheet and become a query — which means any date range works, not just today, and the numbers can never disagree with the underlying entries.

Two of your stated requirements shape this directly. **Bilingual English/Urdu** means every label is a translation key from the first line of code rather than retrofitted later, the layout handles right-to-left, and printed receipts carry an Urdu font. **Production-grade hosting** means automated daily backups with point-in-time recovery — the single thing whose absence caused the original failure.

SEQUENCE

How I'd stage the build

Ordered so the factory gets something usable early rather than waiting for the whole system. Each phase is independently deployable.

01 Foundation

Repository, database schema, authentication, the bilingual framework, and the deployment pipeline. Seed the catalogue with the eighteen paper types, their sizes and current rates — the one piece of the old data worth importing.

NO USER-FACING OUTPUT

02 Billing and customer ledger

Customers, all seven sale forms with their pricing intact, receipt numbering, running balances, and the printable receipt with a WhatsApp

send link. Opening balances get entered from the paper register.

REPLACES MANUAL BILLING

03 Stock

Stock levels across rolls, reels and today, automatic deduction when a bill is raised, manual stock-in on delivery, movement history, and low-stock warnings. This is where the new products get added — through the interface, by your brother, without code.

REPLACES THE MANUAL STOCK REGISTER

04 Reports and expenses

Expense entry, the daily report over any date range, outstanding-balance and stock-valuation views, and a dashboard your brother can check from his phone.

REPLACES THE DAILY REPORT

05 Hardening

Verified backup and restore, an audit trail, staff roles and permissions, and a week of use in the factory with fixes as real problems surface.

THE STEP THE FIRST ATTEMPT SKIPPED
